

CSCI3320 – Fundamentals of Machine Learning

Artificial Intelligence

Computer systems capable of performing complex tasks that mimic human intelligence.

Machine Learning

Algorithms whose performance improve as they are exposed to more data over time.

Elements: *patterns, fuzzy definition, data.*

Data Mining

The use of huge amounts of data for properties.

Statistics

The use of data to make inference about an unknown process.

Formalization

Input (features): $x \in \mathcal{X}$; Output: $y \in \mathcal{Y}$;

Target: $f: \mathcal{X} \rightarrow \mathcal{Y}$; Hypothesis: $g: \mathcal{X} \rightarrow \mathcal{Y}$.

Data: $\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$;

AI in Healthcare: Applied to clinical practice such as disease diagnosis, treatment protocol development, drug design, bioinformatics, personalized medicine, patient monitoring.

AI in Autonomous Driving: Help autonomous vehicles understand the surroundings and perform path planning.

AI in Robotics: Robot learning studies techniques for robot to acquire novel skills or adapt to environment. Surgical robots, assembling robots.

AI in Science: Prediction of chemical structure: AlphaFold2, Financial growth predictions, Weather forecasts.

AI in Languages: NLP, ChatGPT.

AI in Games: AlphaGo, NPCs.

How to Learn from Data

(i) Learning algorithm; (ii) Model; (iii) Data

Types of Learning

Supervised Learning: Learns a function that maps an input to an output based on example input-output pairs. (i) Labeled data, (ii) Predict outcome/future, (iii) Direct feedback.

Unsupervised Learning: Learns patterns from unlabeled data. (i) Unlabeled data, (ii) Find hidden structure, (iii) No feedback.

Reinforcement Learning: Learns to make decisions in an environment to maximize the cumulative award. (i) Learn series, (ii) Reward system, (iii) Decision process.

Regression: A statistical process for estimating the relation between dependent variables (outcomes) and independent variables (features).

Perceptron

$$h(\mathbf{x}) = \text{sign} \left(\sum_{i=1}^d \mathbf{w}_i \mathbf{x}_i \right) = \text{sign}(\mathbf{w}^\top \mathbf{x})$$

Difficulty: $\mathcal{H}$ is infinite, hard to let $g(\mathbf{x}_n) = f(\mathbf{x}_n) = y_n$.

PLA Algorithm

Given a linear separable set of data $\mathcal{D}$, the algorithm finds one linear separator in finite number of steps.

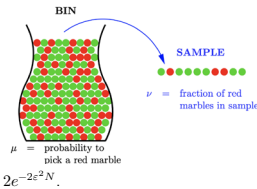
- $\mathbf{w}^{(1)} = \mathbf{0}$
- for $t \leftarrow 1, 2, 3, \dots$
- $(\mathbf{x}^t, y^t) \leftarrow$ any misclassified sample $\text{sign}(\mathbf{w}^{(t)} \cdot \mathbf{x}^t) \neq y^t$
- $\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} + y^t \mathbf{x}^t$

Proof. Assume that there exists $\mathbf{w}^* \in \mathbb{R}^d$ such that $\|\mathbf{w}^*\| = 1$ and $\gamma > 0$ s.t. $y^{(j)}(\mathbf{w}^* \mathbf{x}^{(j)}) > \gamma$, for all $i = 1, 2, \dots, n$, and there exists $R \in \mathbb{R}$ s.t. $\|\mathbf{x}^{(i)}\| \leq R$. Then,

$$\mathbf{w}^{(k+1)} \cdot \mathbf{w}^* = (\mathbf{w}^{(k)} + y^{(j)} \mathbf{x}^{(j)}) \cdot \mathbf{w}^* > \mathbf{w}^{(k)} \cdot \mathbf{w}^* + \gamma.$$

Hence, $k\gamma < \mathbf{w}^{(k+1)} \cdot \mathbf{w}^* \leq \|\mathbf{w}^{(k+1)}\| \|\mathbf{w}^*\| = \|\mathbf{w}^{(k+1)}\|$.
 $\|\mathbf{w}^{(k+1)}\|^2 = \|\mathbf{w}^{(k)} + y^{(j)} \mathbf{x}^{(j)}\|^2 = \|\mathbf{w}^{(k)}\|^2 + 2\mathbf{w}^{(k)} \cdot y^{(j)} \mathbf{x}^{(j)} + \|\mathbf{x}^{(j)}\|^2$
 $\leq \|\mathbf{w}^{(k)}\|^2 + \|\mathbf{x}^{(j)}\|^2 \leq \|\mathbf{w}^{(k)}\|^2 + R^2$, implying that $\|\mathbf{w}^{(k+1)}\|^2 \leq kR^2$.
Therefore, $k^2 \gamma^2 < \|\mathbf{w}^{(k+1)}\|^2 \leq kR^2$ and $k < R^2/\gamma^2$. ■

BIN Model



Pick a sample of N marbles independently.
green: $h(\mathbf{x}) = f(\mathbf{x})$
red: $h(\mathbf{x}) \neq f(\mathbf{x})$
 ν : fraction of red marbles in the sample (*dataset*).
 μ : probability to pick a red marble (whole *distribution*).
 $\mathbb{P}[|\nu - \mu| \geq \varepsilon] \geq 1 -$

$$\mathbf{E}_{\text{out}}(h) = \mathbb{P}_{\mathbf{x}}[h(\mathbf{x}) \neq f(\mathbf{x})], \mathbf{E}_{\text{in}}(h) = \frac{1}{N} \sum_{n=1}^N [h(\mathbf{x}_n) \neq f(\mathbf{x}_n)]$$

Multiple hypotheses: $\mathbb{P}[|\nu - \mu| \geq \varepsilon] \leq 2|\mathcal{H}|e^{-2\varepsilon^2 N} \Rightarrow N =$

$$\frac{2\varepsilon^2}{\ln \frac{2M}{\delta}}.$$

Dichotomy

$h \in \mathcal{H}$ applied to a fixed $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N$

Growth Function

The largest set of dichotomies induced by $\mathcal{H}$:

$$m_{\mathcal{H}}(N) = \max_{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N} |\mathcal{H}(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N)|$$

Break Point for $\mathcal{H}$

No k inputs can be shattered by $\mathcal{H}$.

VC-dim = smallest break point - 1.

Examples

2D positive rays $m_{\mathcal{H}}(N) = N + 1$, $k = 2$

2D positive intervals $m_{\mathcal{H}}(N) = \frac{1}{2}N^2 + \frac{1}{2}N + 1$, $k = 3$

Convex sets $m_{\mathcal{H}}(N) = 2^N$, $k = \infty$

$$m_{\mathcal{H}}(N) = O(N^{k-1})$$

dD perceptron $k = d + 1$

Bounding Function

Maximum dichotomies on N points.

$$B(N, k) \leq B(N-1, k) + B(N-1, k-1)$$

Sauer's lemma:

$$\sum_{i=0}^{k-1} \binom{N}{i} \leq \begin{cases} N^{k-1} + 1 \\ \left(\frac{eN}{k}\right)^{k-1} \end{cases}, m_{\mathcal{H}}(N) \leq N^{d_{\text{VC}}} + 1.$$

Vapnik-Chervonenkis Bound

$$\mathbb{P}[|\nu - \mu| \geq \varepsilon] \leq 4m_{\mathcal{H}}(2N)e^{-\varepsilon^2 N/8}, \mathbf{E}_{\text{out}} \leq \mathbf{E}_{\text{in}} + \sqrt{\frac{8}{N} \log \frac{4m_{\mathcal{H}}(2N)}{\delta}}$$

Higher VC dimension $\rightarrow$ Better chance to approximate complex functions

Lower VC dimension $\rightarrow$ Better chance of generalization to out-of-sample data

$$\mathbf{E}_{\text{out}}(g^{(2)}) = \mathbb{E}_{\mathbf{x}} \left[(g^{(2)}(\mathbf{x}) - f(\mathbf{x}))^2 \right].$$

$$\mathbf{E}_{\mathcal{D}}[\mathbf{E}_{\text{out}}(g^{(2)})] = \mathbf{E}_{\mathcal{D}} \left[\mathbb{E}_{\mathbf{x}} \left[(g^{(2)}(\mathbf{x}) - f(\mathbf{x}))^2 \right] \right] = \mathbb{E}_{\mathbf{x}} \left[\mathbf{E}_{\mathcal{D}} \left[(g^{(2)}(\mathbf{x}) - f(\mathbf{x}))^2 \right] \right],$$

$$\bar{g}(\mathbf{x}) = \mathbf{E}_{\mathcal{D}}[g^{(2)}(\mathbf{x})] \approx \frac{1}{K} \sum_{k=1}^K g^{(2),k}(\mathbf{x}), \mathbf{E}_{\mathcal{D}} \left[(g^{(2)}(\mathbf{x}) - f(\mathbf{x}))^2 \right] =$$

$$\mathbf{E}_{\mathcal{D}} \left[(g^{(2)}(\mathbf{x}) - \bar{g}(\mathbf{x}) + \bar{g}(\mathbf{x}) - f(\mathbf{x}))^2 \right] = \mathbf{E}_{\mathcal{D}} \left[(g^{(2)}(\mathbf{x}) - \bar{g}(\mathbf{x}))^2 \right] +$$

$$(\bar{g}(\mathbf{x}) - f(\mathbf{x}))^2 = \text{var}(\mathbf{x}) + \text{bias}(\mathbf{x}). \mathbf{E}_{\mathcal{D}}[\mathbf{E}_{\text{out}}(g^{(2)})] = \text{bias} + \text{var}.$$

Linear Regression

1. Construct the matrix $\mathbf{X} = \begin{bmatrix} -\mathbf{x}_1^\top \\ -\mathbf{x}_2^\top \\ \vdots \\ -\mathbf{x}_N^\top \end{bmatrix}, \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}$

2. $\mathbf{w}_{\text{lin}} \leftarrow (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$

Noisy Target

$$y = \mathbf{w}^* \mathbf{x} + \varepsilon, \mathbb{E}[\varepsilon] = 0, \text{Var}[\varepsilon] = \sigma^2. \mathbf{E}_{\mathcal{D}}[\mathbf{E}_{\text{in}}(\mathbf{w}_{\text{lin}})] = \sigma^2 \left(1 - \frac{d+1}{N}\right)$$

$$\text{Proof: } \hat{y} = \mathbf{X} \mathbf{w}_{\text{lin}} = \mathbf{H} \mathbf{y}, (\mathbf{I} - \mathbf{H})(\mathbf{X} \mathbf{w}^* + \boldsymbol{\varepsilon}) = (\mathbf{I} - \mathbf{H}) \boldsymbol{\varepsilon}$$

$$\mathbf{E}_{\text{in}}(\mathbf{w}_{\text{lin}}) = \frac{1}{N} (\mathbf{y} - \hat{\mathbf{y}})^\top (\mathbf{y} - \hat{\mathbf{y}}) = \frac{1}{N} ((\mathbf{I} - \mathbf{H}) \mathbf{y})^\top ((\mathbf{I} - \mathbf{H}) \mathbf{y}) =$$

$$\frac{1}{N} ((\mathbf{I} - \mathbf{H}) \boldsymbol{\varepsilon})^\top ((\mathbf{I} - \mathbf{H}) \boldsymbol{\varepsilon}) = \frac{1}{N} \boldsymbol{\varepsilon}^\top (\mathbf{I} - \mathbf{H}) \boldsymbol{\varepsilon}, \mathbf{E}_{\mathcal{D}}[\mathbf{E}_{\text{in}}(\mathbf{w}_{\text{lin}})] =$$

$$\frac{1}{N} \mathbf{E}_{\mathcal{D}}[\text{Tr}((\mathbf{I} - \mathbf{H}) \boldsymbol{\varepsilon} \boldsymbol{\varepsilon}^\top)] = \frac{d^2}{N} \text{Tr}(\mathbf{I} - \mathbf{H}) = \sigma^2 \left(1 - \frac{d+1}{N}\right), \mathbf{E}_{\mathcal{D}}[\mathbf{E}_{\text{out}}(\mathbf{w}_{\text{lin}})] = \sigma^2 \left(1 + \frac{d+1}{N}\right).$$

$$\text{Logistic function: } \theta(s) = \frac{e^s}{1+e^s}, \text{Likelihood: } \prod_{n=1}^N \theta(\mathbf{y}_n \mathbf{w}^\top \mathbf{x}_n).$$

$$\mathbf{E}_{\text{in}}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \ln(1 + e^{-\mathbf{y}_n \mathbf{w}^\top \mathbf{x}_n})$$

Logistic Regression (GD)

1. for $t \leftarrow 0, 1, 2, \dots$ do

2. Compute the gradient $\nabla \mathbf{E}_{\text{in}} = -\frac{1}{N} \sum_{n=1}^N \frac{y_n \mathbf{x}_n}{1 + e^{\mathbf{y}_n \mathbf{w}^\top \mathbf{x}_n}}$

3. $\mathbf{w}(t+1) \leftarrow \mathbf{w}(t) - \eta \nabla \mathbf{E}_{\text{in}}$

Polynomial Transformation

$$\phi_1(\mathbf{x}) = (1, x_1, x_2), \phi_2(\mathbf{x}) = (1, x_1, x_2, x_1^2, x_1 x_2, x_2^2), \dots$$

Neural Network

Neuron: basic computational unit

Activation function is a non-linear function that runs on the neurons of the neural network and responsible for mapping the input of the neuron to the output.

Drawbacks of GD: (i) hard to choose learning rate; (ii) may converge on local minima instead of global minimum; (iii) slow computation

Overfitting

When d_{VC} is too large, bad generalization, $\mathbf{E}_{\text{out}} - \mathbf{E}_{\text{in}}$ is large.

More noise can also contribute to overfitting, adds stochastic noise.

Technique: regularization, validation set, early stopping

Regularization

$$\min_{\mathbf{w}} \mathbf{E}_{\text{in}}(\mathbf{w}) \text{ s.t. } \mathbf{w}^\top \mathbf{w} \leq C \Leftrightarrow \min_{\mathbf{w}} \mathbf{E}_{\text{aug}}(\mathbf{w}) = \mathbf{E}_{\text{in}}(\mathbf{w}) + \frac{\lambda}{N} \mathbf{w}^\top \mathbf{w}$$

Validation

$$\mathcal{D} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{val}}, |\mathcal{D}_{\text{train}}| = N - K, |\mathcal{D}_{\text{val}}| = K.$$

$$\mathbf{E}_{\text{val}}(g^-) = \frac{1}{K} \sum_{\mathbf{x}_n \in \mathcal{D}_{\text{val}}} \text{err}(g^-(\mathbf{x}_n), y_n)$$

Validation error is an unbiased estimate of $\mathbf{E}_{\text{out}}$

$$\mathbf{E}_{\text{val}}[\mathbf{E}_{\text{out}}(g^-)] = \frac{1}{K} \sum_{\mathbf{x}_n \in \mathcal{D}_{\text{val}}} \mathbb{E}_{\mathcal{D}}[\text{err}(g^-(\mathbf{x}_n), y_n)] = \frac{1}{K} \sum_{\mathbf{x}_n \in \mathcal{D}_{\text{val}}} \mathbf{E}_{\text{out}}[g^-] = \mathbf{E}_{\text{out}}[g^-].$$

$$\text{Var}_{\mathcal{D}_{\text{val}}}[\mathbf{E}_{\text{out}}(g^-)] = \frac{1}{K} \text{Var}_{\mathcal{D}}[\mathbf{E}_{\text{out}}(g^-)]. \mathbf{E}_{\text{out}}(h) = \mathbf{E}_{\text{in}}(h) + \text{overfit penalty}.$$

Regularization estimates the overfit, Validation estimates the out-sample error

Occam Razor: The simplest model that fits the data is also the most plausible.

Likelihood Model

When $y \sim \mathcal{N}(\mathbf{X} \mathbf{w}, \sigma^2 \mathbf{I})$, $\text{NLL}(\mathbf{w}) = -\sum_{n=1}^N \log\left(\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(y_n - \mathbf{w}^\top \mathbf{x}_n)^2\right)\right)$ gives the solution $\mathbf{w}_{\text{lin}}$.

Maximum A Posteriori Estimation

$$\mathbf{w}_{\text{MAP}} = \text{argmax}_{\mathbf{w}} \ln p(\mathbf{w} | \mathbf{y}, \mathbf{X}) = \text{argmax}_{\mathbf{w}} \ln p(\mathbf{y} | \mathbf{w}, \mathbf{X}) + \ln p(\mathbf{w})$$

$$p(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{\frac{d}{2}} |\boldsymbol{\Sigma}|^{\frac{1}{2}}} \exp\left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})\right)$$

$$\log p(\mathbf{y} = c | \mathbf{x}, \boldsymbol{\theta}) = \log \pi_c - \frac{1}{2} \log |2\pi \boldsymbol{\Sigma}_c| - \frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_c)^\top \boldsymbol{\Sigma}_c^{-1} (\mathbf{x} - \boldsymbol{\mu}_c) + \text{const.}$$

$$\text{Cat}(\mathbf{y} | \boldsymbol{\theta}) = \prod_{c=1}^C \theta_c^{y^{(c)}}, \log p(\mathcal{D} | \boldsymbol{\theta}) = \left[\sum_{n=1}^N \sum_{c=1}^C I(y_n = c) \log \pi_c \right] +$$

$$\sum_{c=1}^C \left[\sum_{n: y_n=c} \log \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c) \right], \hat{\boldsymbol{\mu}}_c = \frac{1}{N_c} \sum_{n: y_n=c} \mathbf{x}_n, \hat{\boldsymbol{\Sigma}}_c = \frac{1}{N_c} \sum_{n: y_n=c} (\mathbf{x}_n - \hat{\boldsymbol{\mu}}_c)(\mathbf{x}_n - \hat{\boldsymbol{\mu}}_c)^\top$$

Fisher's LDA for Two Classes

$$\boldsymbol{\mu}_1 = \frac{1}{N_1} \sum_{n: y_n=1} \mathbf{x}_n, \boldsymbol{\mu}_2 = \frac{1}{N_2} \sum_{n: y_n=2} \mathbf{x}_n$$

Let $\mathbf{m}_k = \mathbf{w}^\top \boldsymbol{\mu}_k$ be the projection of each mean onto the line, $z_n = \mathbf{w}^\top \mathbf{z}_n$ be the projection of the data onto the line. The variance of the projected points is proportional to

$$s_k^2 = \sum_{n: y_n=k} (z_n - m_k)^2.$$

Goal: Find $\mathbf{w}$ s.t. $J(\mathbf{w}) = \frac{(m_1 - m_2)^2}{s_1^2 + s_2^2}$ is maximized, i.e. $J(\mathbf{w}) =$

$$\frac{\mathbf{w}^\top \mathbf{S}_B \mathbf{w}}{\mathbf{w}^\top \mathbf{S}_W \mathbf{w}}, \text{ where } \mathbf{S}_B = (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^\top \text{ is the between-class}$$

$$\text{scatter matrix and } \mathbf{S}_W = \sum_{n: y_n=1} (\mathbf{x}_n - \boldsymbol{\mu}_1)(\mathbf{x}_n - \boldsymbol{\mu}_1)^\top +$$

$$\sum_{n: y_n=2} (\mathbf{x}_n - \boldsymbol{\mu}_2)(\mathbf{x}_n - \boldsymbol{\mu}_2)^\top \text{ is the within-class scatter matrix.}$$

The maximum of J is given by $\mathbf{S}_B \mathbf{w} = \lambda \mathbf{S}_W \mathbf{w}, \mathbf{w} \propto \mathbf{S}_W^{-1} (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)$

Clustering

Grouping similar data points into clusters, and scatter dissimilar data points across clusters.

K-means

Find cluster centers $\mathbf{m}$ and assignments $\mathbf{r}$ to minimize the sum of squared distances of data points $\{\mathbf{x}^{(n)}\}$ to their assigned cluster centers:

$$\min_{\{\mathbf{m}\}, \{\mathbf{r}\}} J(\{\mathbf{m}\}, \{\mathbf{r}\}) = \min_{\{\mathbf{m}\}, \{\mathbf{r}\}} \sum_{n=1}^N \sum_{k=1}^K r_k^{(n)} \|\mathbf{m}_k - \mathbf{x}^{(n)}\|^2, \text{ s.t. } \sum_k r_k^{(n)} = 1, \forall n, \text{ where } r_k^{(n)} \in \{0, 1\}, \forall k, n$$

1. Set K cluster means $\mathbf{m}_1, \mathbf{m}_2, \dots, \mathbf{m}_K$ to random values

2. Repeat

3. Assignment: Each data point $\mathbf{x}^{(n)}$ assigned to nearest mean. $\hat{k}^{(n)} = \text{argmin}_k d(\mathbf{m}_k, \mathbf{x}^{(n)})$; Responsibilities:

$$r_k^{(n)} = 1 \Leftrightarrow \hat{k}^{(n)} = k$$

4. Refitting: Model parameters, means are adjusted to match sample means of data points they are responsible for:

$$\mathbf{m}_k = \frac{\sum_n r_k^{(n)} \mathbf{x}^{(n)}}{\sum_n r_k^{(n)}}.$$

5. until convergence (no cluster assignment changes)

Gaussian Mixture Model (GMM)

Idea: maximize the probability to reproduce the same distribution of data observed

$$p(\mathbf{x} | \boldsymbol{\pi}_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k), \text{ MLE maximizes } \sum_{n=1}^N \ln \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

EM Algorithm: Softening of binary latent variables

$$\ln p(\mathbf{X}, \mathbf{Z} | \boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \sum_{n=1}^N \sum_{k=1}^K z_{nk} (\ln \pi_k + \ln \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k))$$

1. E-step: Compute the posterior probability that each Gaussian generates each datapoint

$$\gamma_k^{(n)} = p(z^{(n)} = k | \mathbf{x}^{(n)}; \boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{\pi_k \mathcal{N}(\mathbf{x}^{(n)} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}^{(n)} | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

2. M-step: change the parameters of each Gaussian to maximize the probability that it would generate the data it is responsible for

$$\frac{\partial \ln p(\mathbf{X}, \boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma})}{\partial \boldsymbol{\mu}_k} = \sum_{n=1}^N \frac{\pi_k \mathcal{N}(\mathbf{x}^{(n)} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}^{(n)} | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)} \boldsymbol{\Sigma}_k (\mathbf{x}^{(n)} - \boldsymbol{\mu}_k) = 0.$$

$$\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_{n=1}^N \frac{\pi_k \mathcal{N}(\mathbf{x}^{(n)} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}^{(n)} | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)} \mathbf{x}^{(n)},$$

$$\boldsymbol{\Sigma}_k = \frac{1}{N_k} \sum_{n=1}^N \frac{\pi_k \mathcal{N}(\mathbf{x}^{(n)} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}^{(n)} | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)} (\mathbf{x}^{(n)} - \boldsymbol{\mu}_k)(\mathbf{x}^{(n)} - \boldsymbol{\mu}_k)^\top,$$

$$N_k = \sum_{n=1}^N \frac{\pi_k \mathcal{N}(\mathbf{x}^{(n)} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}^{(n)} | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

3. M-step: change the parameters of each Gaussian to maximize the probability